University of Asia Pacific

Department of Computer Science and Engineering

Course Code : CSE 450 (VLSI Lab)

Final Project | Spring 2026

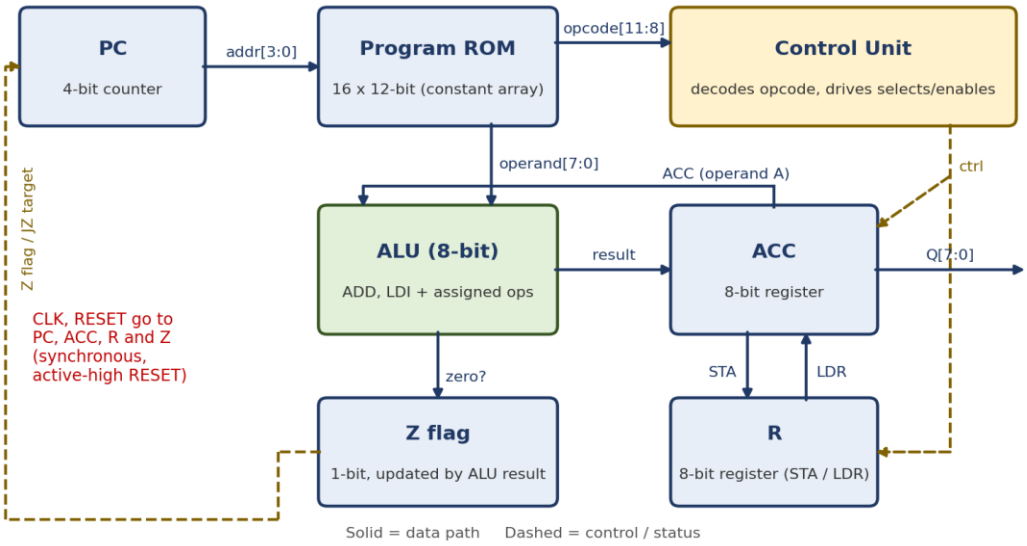
Total Marks	50 (Viva → 30, Report & VrC → 10, Presentation → 10)		
Student Name	Md.Seeman Islam Seemanto	**ID**	22201095

Final Project: Mini 8-bit Accumulator CPU

Objective. Design and implement a small single-cycle 8-bit accumulator CPU in VHDL. The CPU supports only five instructions: two are mandatory for every student, and the remaining three are assigned to you according to your Student ID (see the “Instruction Set Assignment” page: the last digit of your Student ID decides your set). You must also verify the design with a testbench that runs the program given for your set in Appendix B.

1. System Architecture

The CPU is built from the blocks shown below. Every sequential block (PC, ACC, R, Z) is clocked on the rising edge of CLK and cleared by a synchronous, active-high RESET.



Block	Description
PC	4-bit program counter. Holds the address of the instruction being executed. After every instruction it becomes PC + 1, except when JZ is taken.
Program ROM	16 locations x 12 bits, written as a constant array in VHDL. It is read combinationally using PC (no clock needed). Unused locations contain 0x000 (NOP).
Control Unit	Decodes opcode[11:8] and generates the ALU function select and the load/enable signals for ACC, R, Z and PC.
ALU	8-bit combinational unit working on ACC and the 8-bit operand. Only the operations of your own instruction set are required.
ACC, R, Z	ACC: 8-bit accumulator, also the CPU output Q[7:0]. R: 8-bit holding register (only if your set has STA / LDR). Z: 1-bit flag, 1 when the value written to ACC is 0x00.

2. Instruction Format

Each instruction is 12 bits: **instr[11:8] = opcode** (4 bits) and **instr[7:0] = operand** (8-bit immediate). Every instruction takes exactly one clock cycle (fetch, decode and execute happen in the same cycle, the result is written at the rising edge).

Mandatory instructions (all students):

Mnemonic	Opcode	Operation	Flags
LDI imm	0001	ACC ← imm	Z updated
ADD imm	0010	ACC ← ACC + imm (8-bit, carry discarded)	Z updated

Instruction pool (you implement only the three assigned to you):

Mnemonic	Opcode	Operation	Flags
SUB imm	0011	ACC ← ACC − imm (modulo 256)	Z updated
AND imm	0100	ACC ← ACC AND imm	Z updated
OR imm	0101	ACC ← ACC OR imm	Z updated
XOR imm	0110	ACC ← ACC XOR imm	Z updated
NOT	0111	ACC ← NOT ACC (operand ignored)	Z updated
SHL	1000	ACC ← ACC shifted left by 1, 0 enters LSB (operand ignored)	Z updated
SHR	1001	ACC ← ACC shifted right by 1, 0 enters MSB (operand ignored)	Z updated
STA	1010	R ← ACC (operand ignored, ACC unchanged)	Z unchanged
LDR	1011	ACC ← R (operand ignored)	Z updated
JZ addr	1100	If Z = 1 then PC ← operand[3:0], else PC ← PC + 1	Z unchanged

Note: if your set contains LDR it also contains STA. Any opcode not in your set (including 0000) behaves as NOP: PC ← PC + 1 and nothing else changes.

3. Reset Behaviour

RESET = 1 at a rising edge: PC ← 0, ACC ← 0x00, R ← 0x00, Z ← 1 (RESET has priority over everything). **RESET = 0 at a rising edge:** execute the instruction at ROM[PC]. Between clock edges all registers keep their values.

4. Implementation Requirements

- Use VHDL with IEEE.STD_LOGIC_1164.ALL (IEEE.NUMERIC_STD is allowed for arithmetic).
- Modular design: separate entities for PC, Program ROM, Control Unit, ALU and ACC/R/Z, connected in a top-level CPU entity.
- RESET must be synchronous and active-high. All state changes must occur on the rising edge of CLK.
- The top level must expose CLK, RESET, Q[7:0] (the ACC), the Z flag and, for checking, PC[3:0].
- Implement only the five instructions of your own set. The ROM must hold exactly the Appendix B program of your set; the testbench must generate a clock, apply RESET for one rising edge, and run until the last instruction has executed.

Instruction Set Assignment

How to find your set: take the **last digit of your Student ID** (ID mod 10). That digit is your set number in the table below. Example: ID 22201014 → last digit 4 → Set 4.

Every student implements the two mandatory instructions **LDI** and **ADD** plus the three assigned instructions of his/her set. Total = 5 instructions.

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
0	**Set 0**	LDI, ADD	AND, JZ, NOT	LDI, ADD, AND, JZ, NOT
1	**Set 1**	LDI, ADD	SUB, AND, OR	LDI, ADD, SUB, AND, OR
2	**Set 2**	LDI, ADD	XOR, NOT, SHL	LDI, ADD, XOR, NOT, SHL
3	**Set 3**	LDI, ADD	SHR, SUB, XOR	LDI, ADD, SHR, SUB, XOR
4	**Set 4**	LDI, ADD	STA, LDR, AND	LDI, ADD, STA, LDR, AND

Last digit of ID	Set	Mandatory	Assigned	Complete instruction set of the CPU
5	**Set 5**	LDI, ADD	OR, NOT, SHR	LDI, ADD, OR, NOT, SHR
6	**Set 6**	LDI, ADD	SUB, JZ, SHL	LDI, ADD, SUB, JZ, SHL
7	**Set 7**	LDI, ADD	AND, XOR, SHR	LDI, ADD, AND, XOR, SHR
8	**Set 8**	LDI, ADD	STA, LDR, SUB	LDI, ADD, STA, LDR, SUB
9	**Set 9**	LDI, ADD	NOT, SHL, JZ	LDI, ADD, NOT, SHL, JZ

Appendix B: Required Test Program and Expected Results

Store the program of your set (Appendix B tables are ordered by set number 0–9) in the Program ROM. Apply RESET for one rising edge, then run the CPU for one clock cycle per executed instruction. The ACC, Z, R and next-PC columns show the values after the corresponding rising edge. Addresses that are not listed contain 0x000 (NOP). Operands are in hexadecimal. For NOT, SHL, SHR, STA and LDR the operand field is 0x00.

ROM encoding: instr[11:0] = opcode (4 bit) & operand (8 bit). Example: LDI 0x3C = 0001 00111100 = 0x13C.

Set 0: LDI, ADD, AND, JZ, NOT

ROM contents: 0: LDI 0xA5 → 0x1A5 | 1: AND 0x5A → 0x45A | 2: JZ 0x05 → 0xC05 | 3: ADD 0x01 → 0x201 | 4: ADD 0x01 → 0x201 | 5: NOT → 0x700 | 6: AND 0x0F → 0x40F | 7: JZ 0x00 → 0xC00 | 8: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xA5	0xA5	0	10100101	1
2	1	AND 0x5A	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	NOT	0xFF	0	11111111	6
5	6	AND 0x0F	0x0F	0	00001111	7
6	7	JZ 0x00 (not taken)	0x0F	0	00001111	8
7	8	ADD 0x01	0x10	0	00010000	9

Set 1: LDI, ADD, SUB, AND, OR

ROM contents: 0: LDI 0x3C → 0x13C | 1: ADD 0x0F → 0x20F | 2: SUB 0x0B → 0x30B | 3: AND 0xF0 → 0x4F0 | 4: OR 0x0A → 0x50A | 5: SUB 0x4A → 0x34A | 6: SUB 0x01 → 0x301

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x3C	0x3C	0	00111100	1
2	1	ADD 0x0F	0x4B	0	01001011	2
3	2	SUB 0x0B	0x40	0	01000000	3
4	3	AND 0xF0	0x40	0	01000000	4
5	4	OR 0x0A	0x4A	0	01001010	5
6	5	SUB 0x4A	0x00	1	00000000	6
7	6	SUB 0x01	0xFF	0	11111111	7

Set 2: LDI, ADD, XOR, NOT, SHL

ROM contents: 0: LDI 0x55 → 0x155 | 1: XOR 0xFF → 0x6FF | 2: NOT → 0x700 | 3: SHL → 0x800 | 4: ADD 0x56 → 0x256 | 5: XOR 0x81 → 0x681 | 6: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x55	0x55	0	01010101	1
2	1	XOR 0xFF	0xAA	0	10101010	2
3	2	NOT	0x55	0	01010101	3
4	3	SHL	0xAA	0	10101010	4
5	4	ADD 0x56	0x00	1	00000000	5
6	5	XOR 0x81	0x81	0	10000001	6
7	6	SHL	0x02	0	00000010	7

Set 3: LDI, ADD, SHR, SUB, XOR

ROM contents: 0: LDI 0x80 → 0x180 | 1: SHR → 0x900 | 2: SUB 0x10 → 0x310 | 3: ADD 0x0F → 0x20F | 4: XOR 0x3F → 0x63F | 5: ADD 0x07 → 0x207 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x80	0x80	0	10000000	1
2	1	SHR	0x40	0	01000000	2
3	2	SUB 0x10	0x30	0	00110000	3
4	3	ADD 0x0F	0x3F	0	00111111	4
5	4	XOR 0x3F	0x00	1	00000000	5
6	5	ADD 0x07	0x07	0	00000111	6
7	6	SHR	0x03	0	00000011	7

Set 4: LDI, ADD, STA, LDR, AND

ROM contents: 0: LDI 0x1E → 0x11E | 1: STA → 0xA00 | 2: ADD 0x22 → 0x222 | 3: AND 0x60 → 0x460 | 4: LDR → 0xB00 | 5: ADD 0x01 → 0x201 | 6: AND 0x0F → 0x40F | 7: STA → 0xA00 | 8: LDI 0x00 → 0x100 | 9: LDR → 0xB00

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x1E	0x1E	0	0x00	00011110	1
2	1	STA	0x1E	0	0x1E	00011110	2
3	2	ADD 0x22	0x40	0	0x1E	01000000	3
4	3	AND 0x60	0x40	0	0x1E	01000000	4
5	4	LDR	0x1E	0	0x1E	00011110	5
6	5	ADD 0x01	0x1F	0	0x1E	00011111	6
7	6	AND 0x0F	0x0F	0	0x1E	00001111	7
8	7	STA	0x0F	0	0x0F	00001111	8
9	8	LDI 0x00	0x00	1	0x0F	00000000	9
10	9	LDR	0x0F	0	0x0F	00001111	10

Set 5: LDI, ADD, OR, NOT, SHR

ROM contents: 0: LDI 0x0F → 0x10F | 1: OR 0xA0 → 0x5A0 | 2: NOT → 0x700 | 3: SHR → 0x900 | 4: ADD 0xD8 → 0x2D8 | 5: OR 0x81 → 0x581 | 6: SHR → 0x900

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x0F	0x0F	0	00001111	1
2	1	OR 0xA0	0xAF	0	10101111	2
3	2	NOT	0x50	0	01010000	3
4	3	SHR	0x28	0	00101000	4
5	4	ADD 0xD8	0x00	1	00000000	5
6	5	OR 0x81	0x81	0	10000001	6
7	6	SHR	0x40	0	01000000	7

Set 6: LDI, ADD, SUB, JZ, SHL

ROM contents: 0: LDI 0x02 → 0x102 | 1: SUB 0x01 → 0x301 | 2: JZ 0x07 → 0xC07 | 3: SUB 0x01 → 0x301 | 4: JZ 0x07 → 0xC07 | 5: ADD 0xAA → 0x2AA | 6: ADD 0xAA → 0x2AA | 7: LDI 0x21 → 0x121 | 8: SHL → 0x800 | 9: ADD 0x01 → 0x201

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0x02	0x02	0	00000010	1
2	1	SUB 0x01	0x01	0	00000001	2
3	2	JZ 0x07 (not taken)	0x01	0	00000001	3
4	3	SUB 0x01	0x00	1	00000000	4
5	4	JZ 0x07 (taken)	0x00	1	00000000	7
6	7	LDI 0x21	0x21	0	00100001	8
7	8	SHL	0x42	0	01000010	9
8	9	ADD 0x01	0x43	0	01000011	10

Set 7: LDI, ADD, AND, XOR, SHR

ROM contents: 0: LDI 0xF3 → 0x1F3 | 1: AND 0x3C → 0x43C | 2: XOR 0x11 → 0x611 | 3: SHR → 0x900 | 4: ADD 0x30 → 0x230 | 5: XOR 0x40 → 0x640

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xF3	0xF3	0	11110011	1
2	1	AND 0x3C	0x30	0	00110000	2
3	2	XOR 0x11	0x21	0	00100001	3
4	3	SHR	0x10	0	00010000	4
5	4	ADD 0x30	0x40	0	01000000	5
6	5	XOR 0x40	0x00	1	00000000	6

Set 8: LDI, ADD, STA, LDR, SUB

ROM contents: 0: LDI 0x64 → 0x164 | 1: STA → 0xA00 | 2: SUB 0x14 → 0x314 | 3: ADD 0x0A → 0x20A | 4: STA → 0xA00 | 5: SUB 0x5A → 0x35A | 6: LDR → 0xB00 | 7: SUB 0x60 → 0x360

Cycle	PC	Instruction executed	ACC (hex)	Z	R (hex)	ACC (bin)	Next PC
0	-	RESET = 1	0x00	1	0x00	00000000	0
1	0	LDI 0x64	0x64	0	0x00	01100100	1
2	1	STA	0x64	0	0x64	01100100	2
3	2	SUB 0x14	0x50	0	0x64	01010000	3
4	3	ADD 0x0A	0x5A	0	0x64	01011010	4
5	4	STA	0x5A	0	0x5A	01011010	5
6	5	SUB 0x5A	0x00	1	0x5A	00000000	6
7	6	LDR	0x5A	0	0x5A	01011010	7
8	7	SUB 0x60	0xFA	0	0x5A	11111010	8

Set 9: LDI, ADD, NOT, SHL, JZ

ROM contents: 0: LDI 0xFF → 0x1FF | 1: NOT → 0x700 | 2: JZ 0x05 → 0xC05 | 3: ADD 0x77 → 0x277 | 4: ADD 0x77 → 0x277 | 5: ADD 0x11 → 0x211 | 6: SHL → 0x800 | 7: JZ 0x00 → 0xC00 | 8: SHL → 0x800

Cycle	PC	Instruction executed	ACC (hex)	Z	ACC (binary)	Next PC
0	-	RESET = 1	0x00	1	00000000	0
1	0	LDI 0xFF	0xFF	0	11111111	1
2	1	NOT	0x00	1	00000000	2
3	2	JZ 0x05 (taken)	0x00	1	00000000	5
4	5	ADD 0x11	0x11	0	00010001	6
5	6	SHL	0x22	0	00100010	7
6	7	JZ 0x00 (not taken)	0x22	0	00100010	8
7	8	SHL	0x44	0	01000100	9

Submission Guidelines

Submit your VHDL source files, testbench and a project report within the 2-week period. Include this project description (question, assignment page and Appendix B) at the front of your report and start your own report on a new page after it. The report must include, but is not limited to, the following points.

- Your Student ID, assigned set number and the three assigned instructions (highlighted in yellow color)
- Attach your entity code (all modules), testbench code and output visuals (waveform with PC, ACC, Z and R if used)
- Challenges faced during the experiment, and conclusion

Student ID Last Digit: 5
Assigned Instruction Set: Set 5
Assigned Instructions: LDI, ADD, OR, NOT, SHR

Project Title

Design and Simulation of a Mini 8-bit Accumulator CPU Using Verilog HDL

Project Description

This project involves the design, Verilog HDL modeling, synthesis, and behavioral verification of a single-cycle Mini 8-Bit Accumulator-Based CPU. The architecture is optimized to execute instructions in a single clock cycle through integrated combinational instruction fetch and decode logic combined with synchronous rising-edge state updates.

Assigned Instructions

- LDI (Opcode → 0001)
- ADD (Opcode → 0010)
- OR (Opcode → 0101)
- NOT (Opcode → 0111)
- SHR (Opcode → 1001)

Project Objective

This project models a simple 8-bit accumulator CPU in Verilog HDL. Built from modular components—including a 4-bit PC, 16x8-bit ROM, Control Unit, ALU, ACC register, and Zero Flag—the processor fetches and executes ROM-based instructions. The modular structure simplifies debugging, simulation, and long-term maintenance.

System Architecture

The CPU consists of five interconnected submodules operating synchronously on the rising edge of CLK with a synchronous, active-high RESET:

1. **Program Counter (PC)**: A 4-bit counter holding the memory address of the instruction being executed (PC[3:0]). Increments sequentially by 1 every clock cycle.
2. **Program ROM (ROM)**: A 16×12-bit combinational memory array storing the pre-loaded program instructions. Reads instruction words asynchronously using PC[3:0].
3. **Control Unit (Control Unit)**: Decodes opcode[11:8] to produce control signals (alu_op[2:0] and load_acc).
4. **Arithmetic Logic Unit (ALU)**: Combines the Accumulator output ($Q[7:0]$) and immediate operand (operand[7:0]) to calculate the target result.
5. **Accumulator and Zero Register (ACC & Z)**: An 8-bit register holding the central state $Q[7:0]$ and a 1-bit zero flag (Z) updated whenever load_acc = 1.

Module Interface Pin Configuration

cpu_top:

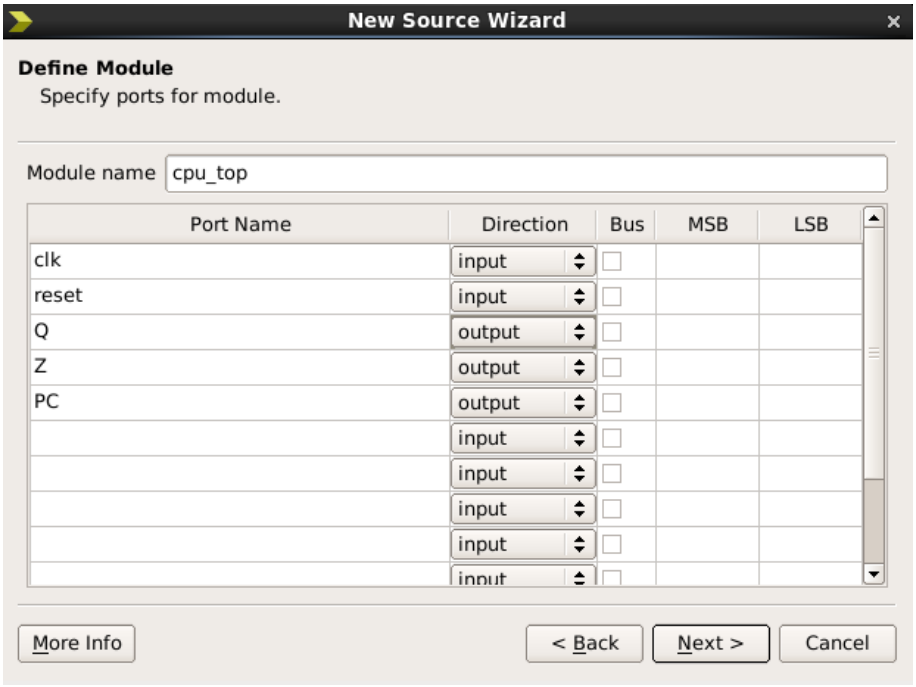


Fig-1: cpu_top pin configuration

pc:

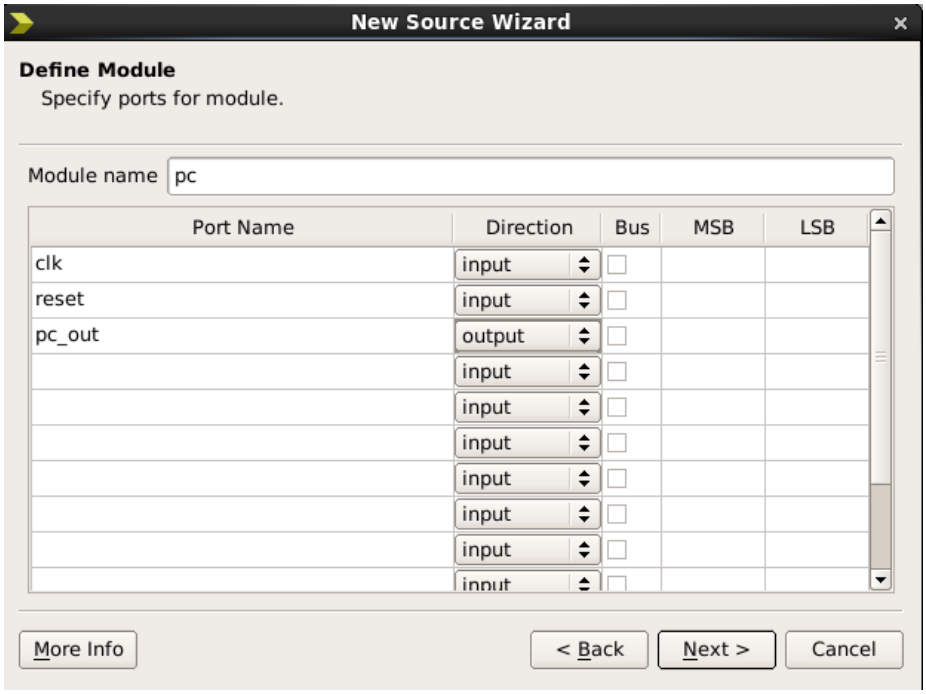


Fig-2: program-counter pin configuration

rom:

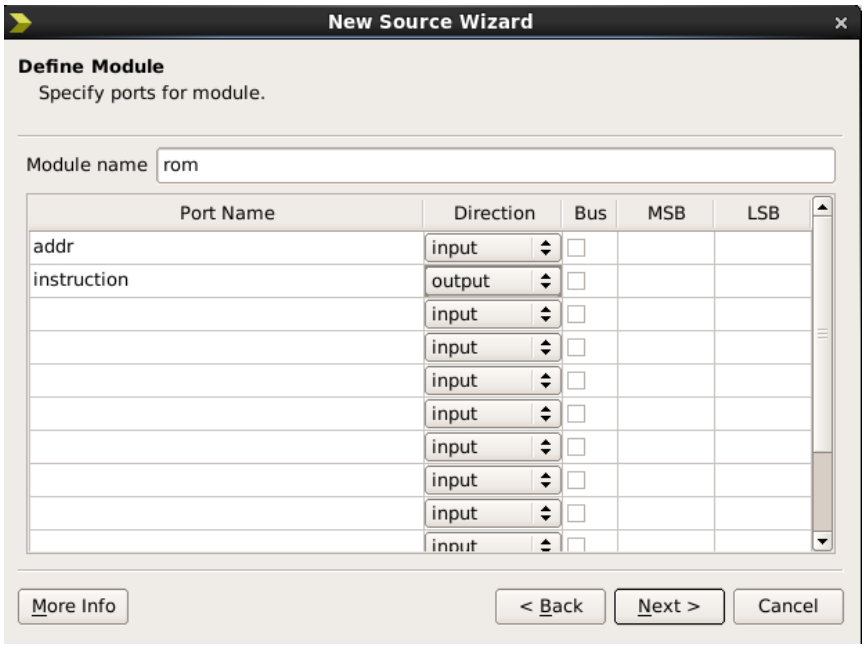


Fig-3: ROM pin configuration

control_unit:

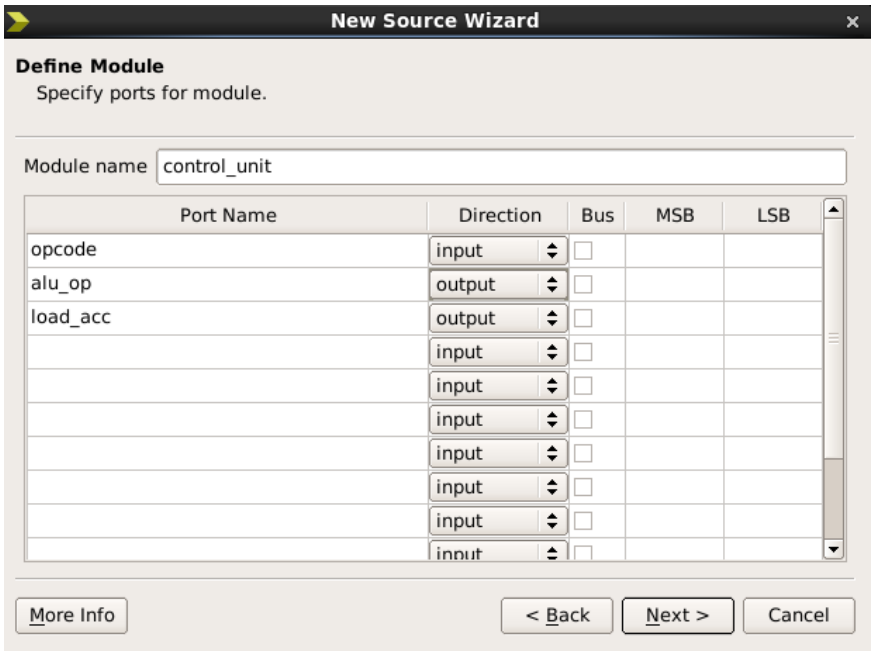


Fig-4: control-unit pin configuration

alu:

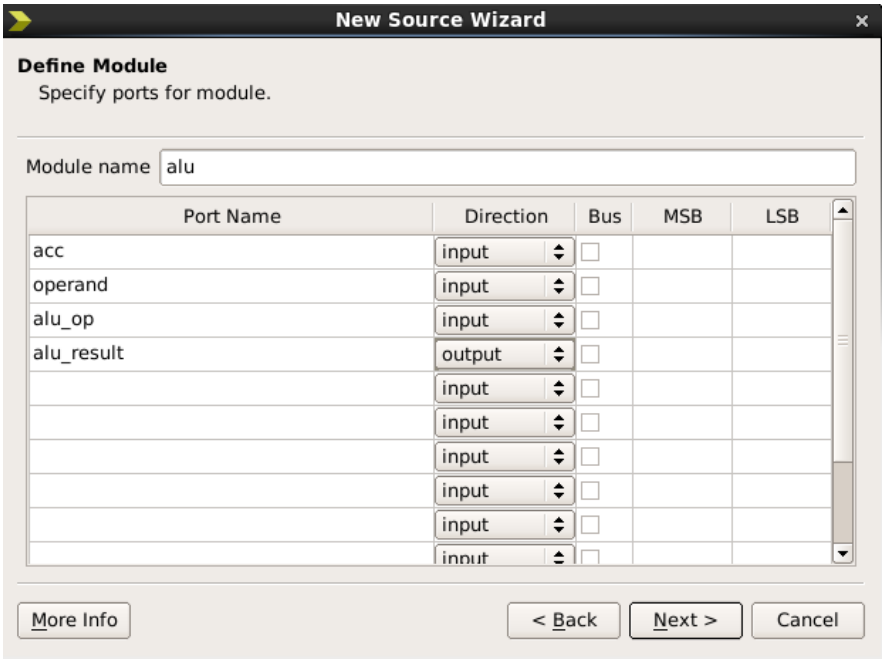


Fig-5: ALU pin configuration

acc & z:

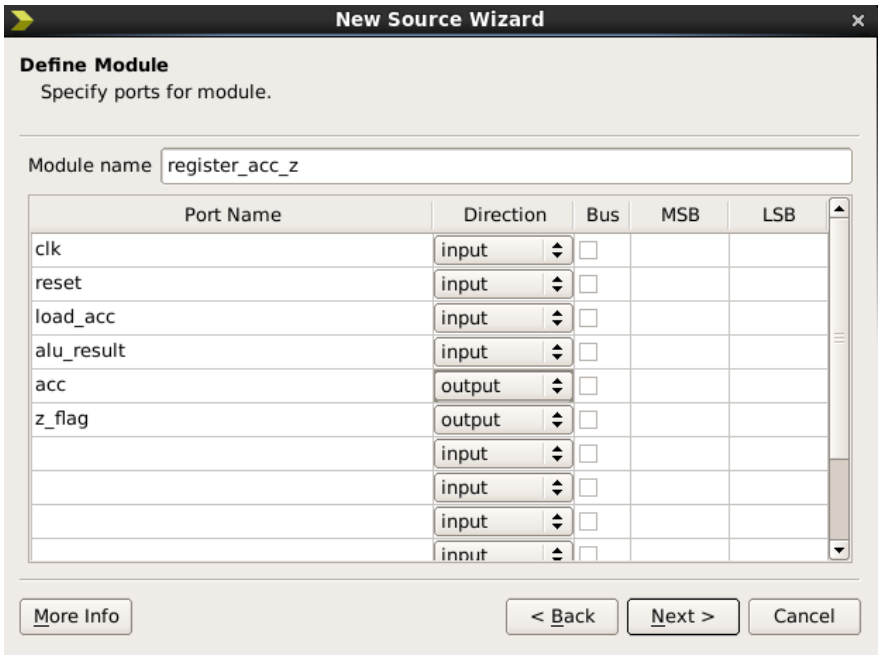


Fig-6: acc and z-flag pin configuration

Verilog Source Codes

pc.v

```
`timescale 1ns / 1ps
module pc (
    input wire clk,
    input wire reset,
    output reg [3:0] pc_out
);

    always @(posedge clk) begin
        if (reset) begin
            pc_out <= 4'b0000;
        end else begin
            pc_out <= pc_out + 1'b1;
        end
    end

endmodule
```

rom.v

```
`timescale 1ns / 1ps

module rom (
    input wire [3:0] addr,
    output reg [11:0] instruction
);

    always @(*) begin
        case (addr)
            4'h0: instruction = 12'h10F; // LDI 0x0F
            4'h1: instruction = 12'h5A0; // OR  0xA0
            4'h2: instruction = 12'h700; // NOT
            4'h3: instruction = 12'h900; // SHR
            4'h4: instruction = 12'h2D8; // ADD 0xD8
            4'h5: instruction = 12'h581; // OR  0x81
            4'h6: instruction = 12'h900; // SHR
            default: instruction = 12'h000; // NOP for unused memory
        endcase
    end

endmodule
```

control_unit.v

```
`timescale 1ns / 1ps
module control_unit (
    input wire [3:0] opcode,
```

```
    output reg [2:0] alu_op,
    output reg load_acc
);

    always @(*) begin
        case (opcode)
            4'b0001: begin // LDI
                alu_op   = 3'b000;
                load_acc = 1'b1;
            end
            4'b0010: begin // ADD
                alu_op   = 3'b001;
                load_acc = 1'b1;
            end
            4'b0101: begin // OR
                alu_op   = 3'b010;
                load_acc = 1'b1;
            end
            4'b0111: begin // NOT
                alu_op   = 3'b011;
                load_acc = 1'b1;
            end
            4'b1001: begin // SHR
                alu_op   = 3'b100;
                load_acc = 1'b1;
            end
            default: begin // NOP / Unused opcodes
                alu_op   = 3'b000;
                load_acc = 1'b0;
            end
        endcase
    end

endmodule
```

alu.v

```
`timescale 1ns / 1ps

module alu (
    input wire [7:0] acc,
    input wire [7:0] operand,
    input wire [2:0] alu_op,
    output reg [7:0] alu_result
);

    always @(*) begin
        case (alu_op)
```

```
            3'b000: alu_result = operand;         // LDI
            3'b001: alu_result = acc + operand;   // ADD
            3'b010: alu_result = acc | operand;   // OR
            3'b011: alu_result = ~acc;            // NOT
            3'b100: alu_result = {1'b0, acc[7:1]};// SHR (Logical Right Shift)
            default: alu_result = 8'h00;
        endcase
    end

endmodule
```

register_acc_z.v

```
`timescale 1ns / 1ps
module register_acc_z (
    input wire clk,
    input wire reset,
    input wire load_acc,
    input wire [7:0] alu_result,
    output reg [7:0] acc,
    output reg z_flag
);

    always @(posedge clk) begin
        if (reset) begin
            acc    <= 8'h00;
            z_flag <= 1'b1;
        end else if (load_acc) begin
            acc    <= alu_result;
            z_flag <= (alu_result == 8'h00) ? 1'b1 : 1'b0;
        end
    end

endmodule
```

cpu_top.v

```
`timescale 1ns / 1ps
module cpu_top (
    input wire clk,
    input wire reset,
    output wire [7:0] Q,
    output wire Z,
    output wire [3:0] PC
);

    // Internal Interconnect Wires
    wire [11:0] instruction;
```

```
    wire [2:0] alu_op;
    wire load_acc;
    wire [7:0] alu_result;

    // 1. Program Counter Instance
    pc pc_inst (
        .clk(clk),
        .reset(reset),
        .pc_out(PC)
    );

    // 2. Program ROM Instance
    rom rom_inst (
        .addr(PC),
        .instruction(instruction)
    );

    // 3. Control Unit Instance
    control_unit cu_inst (
        .opcode(instruction[11:8]),
        .alu_op(alu_op),
        .load_acc(load_acc)
    );

    // 4. ALU Instance
    alu alu_inst (
        .acc(Q),
        .operand(instruction[7:0]),
        .alu_op(alu_op),
        .alu_result(alu_result)
    );

    // 5. Accumulator & Z Flag Register Instance
    register_acc_z acc_z_inst (
        .clk(clk),
        .reset(reset),
        .load_acc(load_acc),
        .alu_result(alu_result),
        .acc(Q),
        .z_flag(Z)
    );

endmodule
```

tb_cpu_top.v

```
`timescale 1ns / 1ps

```

```
module tb_cpu_top;

    // Inputs
    reg clk;
    reg reset;

    // Outputs
    wire [7:0] Q;
    wire Z;
    wire [3:0] PC;

    // Instantiate Unit Under Test (UUT)
    cpu_top uut (
        .clk(clk),
        .reset(reset),
        .Q(Q),
        .Z(Z),
        .PC(PC)
    );

    // Clock Generation (10ns period: 5ns LOW, 5ns HIGH)
    always #5 clk = ~clk;

    initial begin
        // Initialize Signals
        clk = 0;
        reset = 1;

        // Apply synchronous active-high RESET for one rising edge
        #10;
        reset = 0;

        // Run for 8 clock cycles (80ns total) to complete all Set 5
instructions
        #80;

        $stop;
    end

    // Monitor Output in ISim Console
    initial begin
        $display("Time\t\tPC\tRESET\tACC(Hex)\tZ\tACC(Binary)");
        $monitor("%d ns\t%d\t%b\t0x%h\t\t%b\t%b", $time, PC, reset, Q, Z, Q);
    end

endmodule
```

Verilog Source Codes

Reference vs. ISim Observed Waveform Comparison

Cycle	Time (ns)	PC	Executed Instruction	ACC / Q (Hex)	Z Flag	ACC / Q (Binary)	Next PC	Verification Status
0	0 - 10	0	RESET = 1	0x00	1	00000000	0	**Matched**
1	10 - 20	0	LDI 0x0F	0x0F	0	00001111	1	**Matched**
2	20 - 30	1	OR 0xA0	0xAF	0	10101111	2	**Matched**
3	30 - 40	2	NOT	0x50	0	01010000	3	**Matched**
4	40 - 50	3	SHR	0x28	0	00101000	4	**Matched**
5	50 - 60	4	ADD 0xD8	0x00	1	00000000	5	**Matched**
6	60 - 70	5	OR 0x81	0x81	0	10000001	6	**Matched**
7	70 - 80	6	SHR	0x40	0	01000000	7	**Matched**

Detailed Cycle Analysis

1. **Cycle 0 (0 - 10 ns)**: Synchronous RESET = 1 forces Q <= 0x00, Z <= 1, and PC <= 0000.
2. **Cycle 1 (10 - 20 ns)**: LDI 0x0F loads immediate value 0x0F (00001111) into Q. Z becomes 0.
3. **Cycle 2 (20 - 30 ns)**: OR 0xA0 performs bitwise OR (0x0F | 0xA0 = 0xAF / 10101111).
4. **Cycle 3 (30 - 40 ns)**: NOT inverts bits (~0xAF = 0x50 / 01010000).
5. **Cycle 4 (40 - 50 ns)**: SHR shifts 0x50 right by 1 (0x50 >> 1 = 0x28 / 00101000).
6. **Cycle 5 (50 - 60 ns)**: ADD 0xD8 adds 0x28 + 0xD8 = 0x100. The 8-bit register discards the 9th carry bit, storing 0x00 (00000000). Zero flag Z asserts to 1.
7. **Cycle 6 (60 - 70 ns)**: OR 0x81 computes 0x00 | 0x81 = 0x81 (10000001). Z updates to 0.
8. **Cycle 7 (70 - 80 ns)**: SHR shifts 0x81 right by 1 (0x81 >> 1 = 0x40 / 01000000).

The top-level ISim waveform simulation output confirms complete alignment with the official **Appendix B Set 5** expected execution table.

Waveform Output



Fig-7: tb_cpu_top waveform output

Challenges Faced

During the implementation process, I encountered and resolved three main architectural challenges:

First, managing 8-bit arithmetic overflow during addition posed a potential issue. For instance, in Cycle 5, adding *0x28* and *0xD8* yields *0x100*, which requires 9 bits to represent. We resolved this by constraining the ALU output port strictly to 8 bits *([7:0])*, allowing the system to naturally drop the carry-out bit and maintain standard 8-bit register truncation.

Second, updating the Zero Flag synchronously presented a risk of combinational feedback loops when attempting to evaluate *Z* directly from the accumulator's stored output. To eliminate this, we modified *register_acc_z.v* so that Z is calculated directly from alu_result inside the sequential *always @(posedge clk)* block. This preserved clean single-cycle timing without introducing unstable timing loops.

Finally, keeping all operations within a single clock cycle required strict timing alignment across instruction fetching, decoding, and execution. We addressed this by designing both the Program ROM and Control Unit as pure combinational logic using *always @(*)* blocks. This setup allowed the CPU to fetch, decode, and compute the ALU result within a single clock period well before the next rising clock edge.

Conclusion

Successfully designed, synthesized, and verified a single-cycle Mini 8-Bit Accumulator CPU tailored for Set 5 instructions (LDI, ADD, OR, NOT, and SHR) using Verilog HDL within Xilinx ISE. Following modular design practices, we partitioned the architecture into dedicated components for the Program Counter, ROM, Control Unit, ALU, and ACC/Z registers. Finally, behavioral simulation in ISim confirmed that all register outputs, zero flag updates, and program counter states precisely aligned with the expected benchmark behavior outlined in Appendix B.

GitHub Repository

Link